SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Grama Jyothi Billing Program

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

TASK 3 – Define the Constants

Aim

To keep all tariff values in a single place so that a revision is a one-line change.

Concept

- **#define:** textual substitution by the preprocessor; no type, no storage, no scope, not visible in the debugger. Good for compile-time values such as array sizes.
- **const variable:** typed, has scope and an address, visible in the debugger, but in C it is not a constant expression (cannot size a normal array).
- **enum:** named integer constants (int), grouped, visible to the debugger; good for categories, e.g. enum {DOMESTIC, COMMERCIAL}.
- Naming: UPPER_CASE with unit suffixes (RATE2_PAISE), descriptive, one prefix per group.

Complete the Code – TODO 1

```
long paise = FIXED_CHARGE_PAISE;
if (units > SLAB2_UNITS) {
    paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
    units = SLAB2_UNITS;
}
if (units > SLAB1_UNITS) {
    paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
    units = SLAB1_UNITS;
}
paise += (long)units * RATE1_PAISE;
return paise;
```

Output (matches expected):

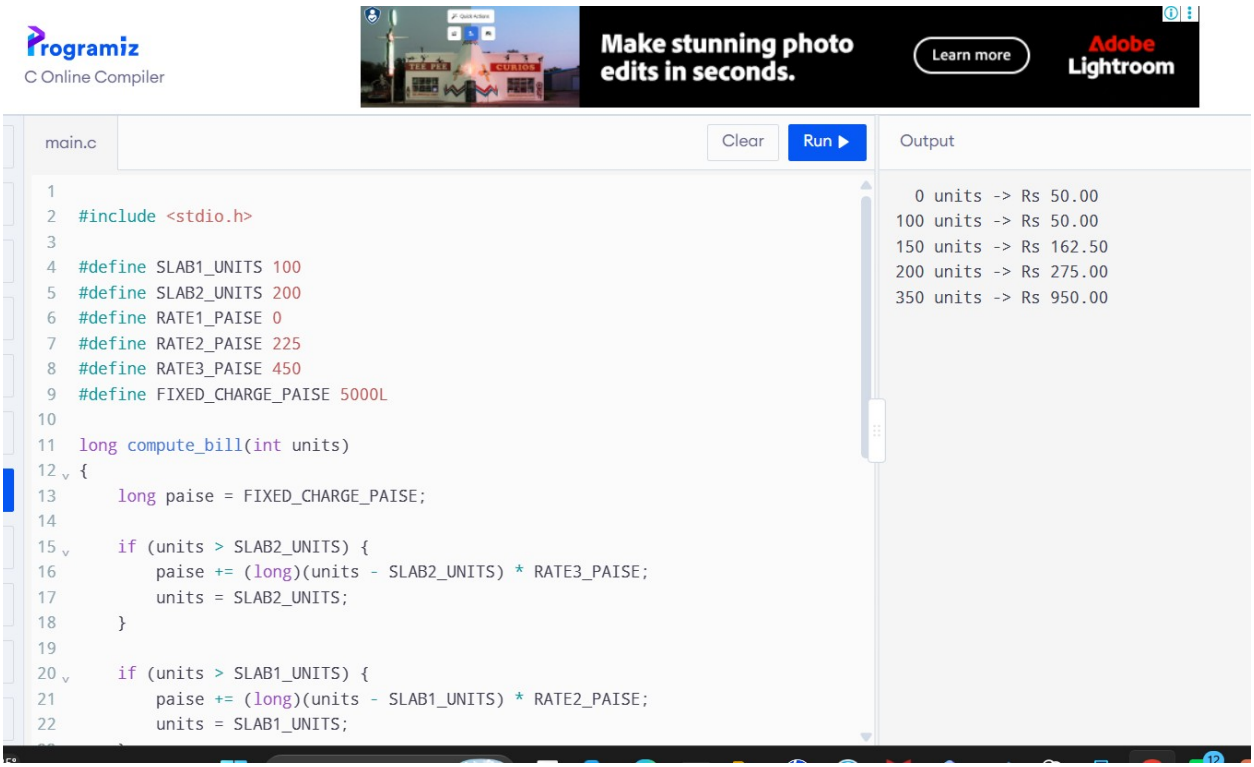
```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

After changing RATE2_PAISE to 230:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 165.00
200 units -> Rs 280.00
```

350 units -> Rs 955.00

Every bill above 100 units rises consistently (50 units × 5 paise = Rs 2.50 at 150; 100 × 5 paise = Rs 5.00 at ≥200), and bills up to 100 units do not change, as expected.



Alternative Concept – const variables or enum

Replace #define RATE2_PAISE 225 with const int RATE2_PAISE = 225; or enum { RATE2_PAISE = 225 }. A const gains type checking and debugger visibility but may occupy 4 bytes of storage (an optimising compiler usually removes it); an enum gives integer constants with no storage and debugger names, but only int type (so 5000L must stay a macro or const long). #define uses no storage. Running time is unchanged: after optimisation all three become immediate operands, O(1).

Result: One set of constants; #define chosen for tariff values, enum for category codes.